UNIVERSITY OF WOLLONGONG IN DUBAI

MDT915 — BLOCKCHAIN IMPLEMENTATION PROJECT

CharityFlow: Transparent Charity Donation Platform

Course: MDT915 Blockchain Practitioner
Weight: 30% of final grade
Type: Group Project (2 students)
Due Date: Session 6 — March 14, 2026, 1:00 PM
Repository: github.com/GuidateTest/charityflow-blockchain

Team Members

Name	Student ID	Role
Seif Sid Ali Maloufi	8718179	Developer (Smart Contracts, Frontend, Web3, IPFS)
Jyldyz Sulaimanova	9105141	Report & Theory (Documentation, Research, Academic Writing)

1. Executive Summary

ChartyFlow is a decentralized application (DApp) that addresses the lack of transparency in charitable donations. Donors can track exactly how their contributions are used through an immutable blockchain ledger. Charities create campaigns, receive donations in cryptocurrency (ETH), and must declare withdrawals with proof before accessing funds. Every transaction—donations, withdrawals, proof submissions, and impact reports—is recorded on-chain and visible to all stakeholders.

The system is deployed on **Sepolia testnet** and supports both local Hardhat development and live testnet transactions. Key features include IPFS integration for document uploads, real-time on-chain activity logs, multi-network support (Hardhat + Sepolia), and a responsive React frontend.

2. Problem Statement

Traditional charity platforms suffer from:

- **Opacity:** Donors cannot verify how their money is spent.
- **Trust issues:** Charities may misuse funds without accountability.
- **Centralization:** Intermediaries control funds and charge high fees.
- **Lack of proof:** Impact claims are often unverifiable.

CharityFlow solves these by recording every donation, withdrawal, and proof of work on the Ethereum blockchain, making charity fully transparent and auditable.

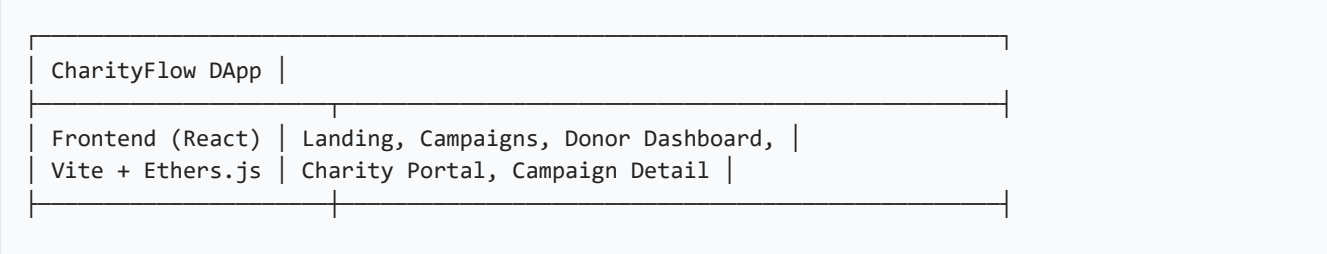
3. Solution Design

3.1 Domain Choice

We selected **Option 2: Transparent Charity Donation Platform** from the assignment, implementing all required features:

- Donation campaigns creation by charities
- Donor contributions in cryptocurrency (ETH)
- Transparent fund tracking (donors see where money goes)
- Impact reporting (charities upload proof of work via IPFS)

3.2 Architecture Overview



```
| Web3 Layer | MetaMask → BrowserProvider → Signer → Contract |
| Ethers.js v6 | Multi-network: Hardhat (31337) / Sepolia (11155111)|
├──────────────────────┼──────────────────────────────────────────────┤
| Smart Contract | CharityDonation.sol (Solidity 0.8.20) |
| CharityDonation | Campaigns, Donations, Proofs, Withdrawals, Reports|
├──────────────────────┼──────────────────────────────────────────────┤
| Storage | IPFS (Pinata) for images, receipts, proofs |
├──────────────────────┼──────────────────────────────────────────────┤
| Blockchain | Hardhat Local / Sepolia Testnet |
| Ethereum | Chain ID: 31337 (local) / 11155111 (Sepolia) |
└──────────────────────┴──────────────────────────────────────────────┘
```

3.3 Data Flow

```
Donor → MetaMask → donate(campaignId, message) → CharityDonation Contract
↓
Funds held in contract escrow
↓
Charity → withdrawFunds(campaignId, amount, purpose, proofHash) → ETH sent
↓
Charity → submitProof(campaignId, title, description, fileHash, amountSpent)
↓
Charity → postImpactReport(campaignId, title, content, mediaHashes)
↓
Admin → verifyProof(proofId) / verifyCharity(address)
```

4. Implementation Details

4.1 Smart Contract (CharityDonation.sol)

Core structures: Campaign, Donation, ProofOfWork, WithdrawalRecord, ImpactReport

Key functions: createCampaign, donate, withdrawFunds, submitProof, postImpactReport, cancelCampaign, claimRefund. Admin: verifyCharity, verifyCampaign, verifyProof, withdrawPlatformFees.

Platform fee: 2% (200 basis points) on each donation.

4.2 Frontend Features

Landing page, campaign list (search, filter, sort), campaign detail with donation form, donor dashboard with on-chain activity log, charity portal (create campaigns, withdraw, submit proofs, impact reports), IPFS upload via Pinata, multi-network support.

Application Screenshots

Figure 1 — Landing Page: Hero section with feature highlights and call-to-action.

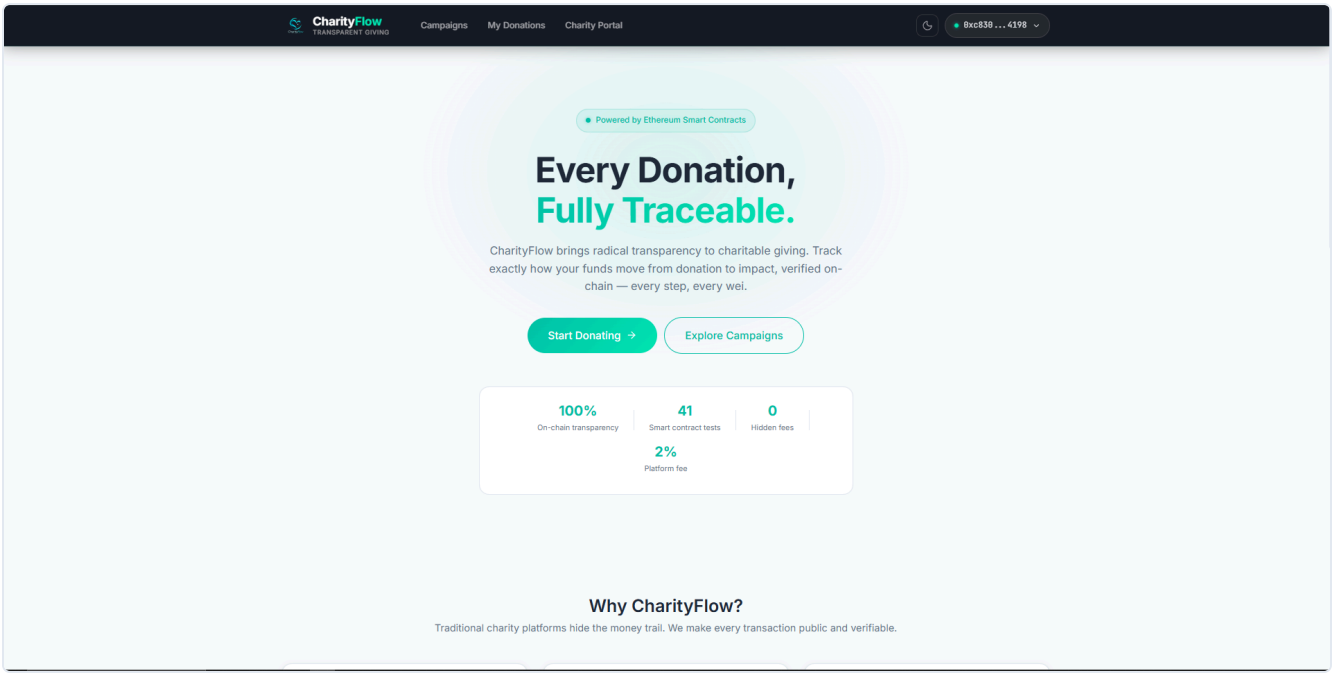


Figure 2 — Campaign List: Browse campaigns with search, filter, and sort.

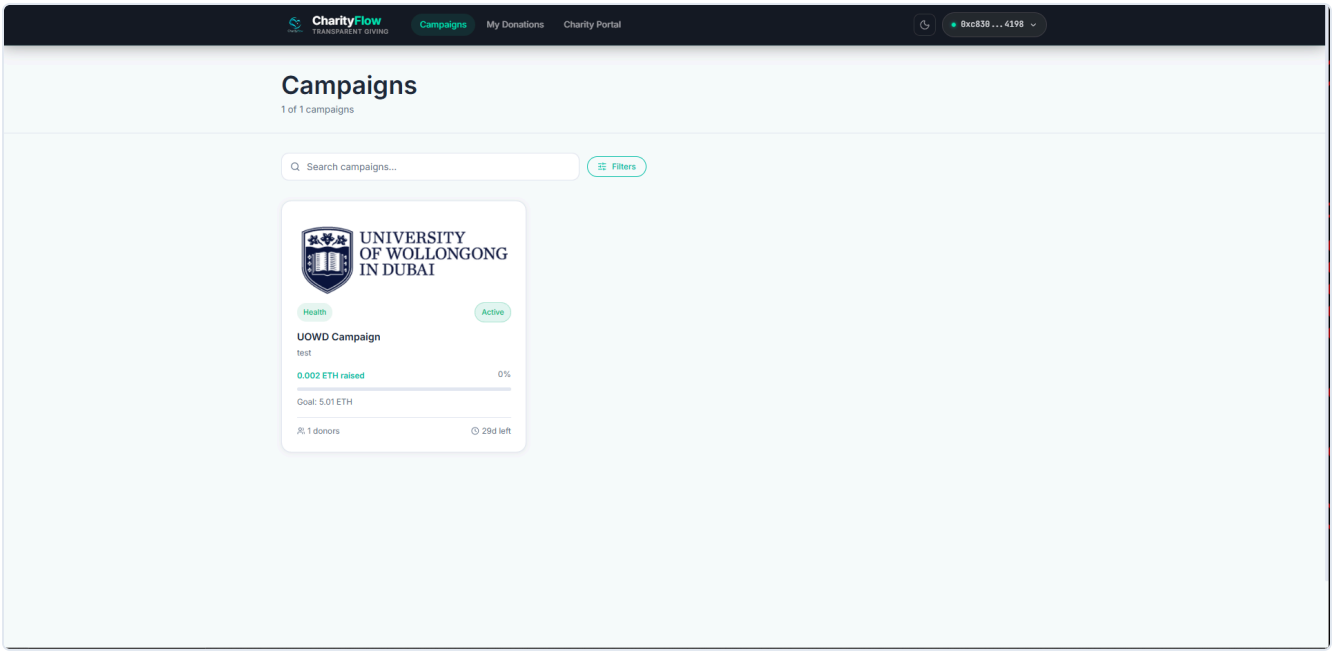


Figure 3 — Campaign Detail: Full campaign info, progress bar, donation form, and Fund Flow.

Figure 4 — Donor Dashboard: My donations with on-chain activity log.

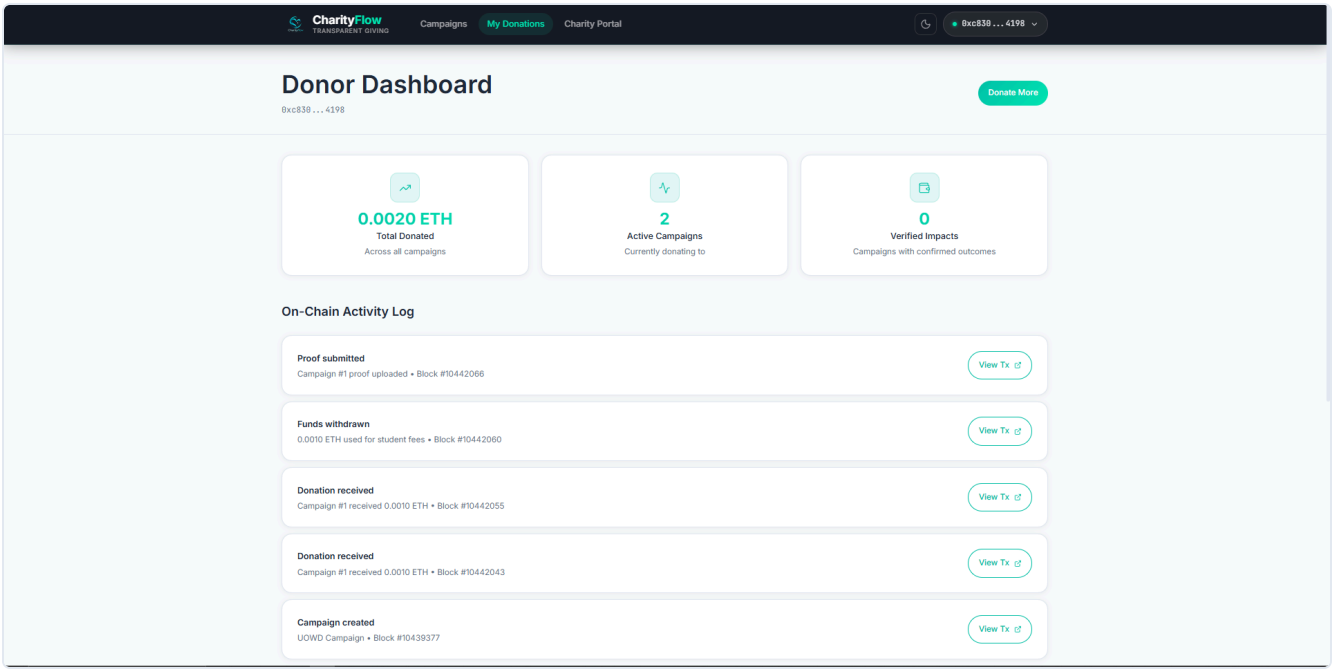
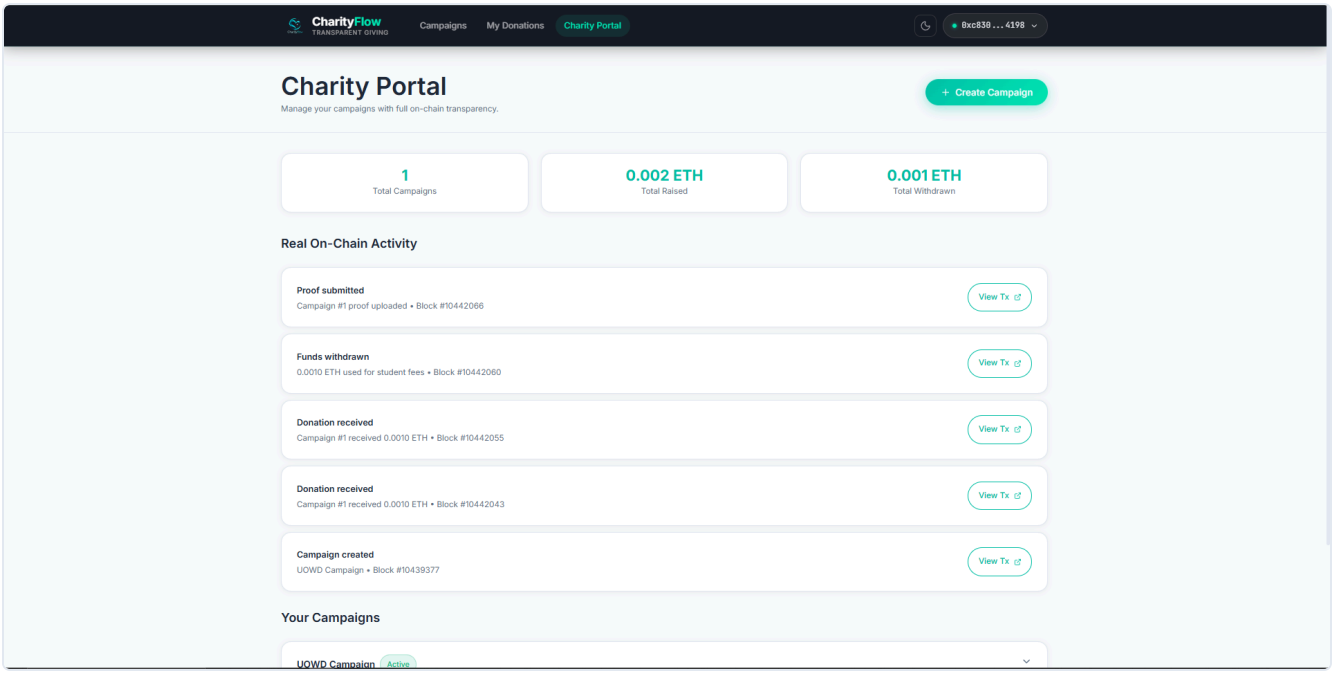


Figure 5 — Charity Portal: Create and manage campaigns.



4.3 IPFS Integration

Pinata API for file uploads. Campaign cover images, proof documents, withdrawal receipts, impact report media. Gateway: https://gateway.pinata.cloud/ipfs/{cid}

5. Technologies Used

Layer	Technology	Purpose
Blockchain	Ethereum (Hardhat / Sepolia)	Immutable ledger
Smart Contracts	Solidity 0.8.20	Business logic
Development	Hardhat 2.22.x	Compile, test, deploy
Testing	Mocha + Chai	41 unit tests
Frontend	React 19 + Vite 4	UI
Web3	Ethers.js v6	Blockchain interaction
Wallet	MetaMask	User authentication
Storage	Pinata (IPFS)	Images, proofs, receipts

6. Prerequisites

- Node.js v18+ (v20 recommended)

- npm 9+
- MetaMask browser extension
- Pinata account (free) for IPFS
- Sepolia test ETH from faucet

7. Installation & Setup

Local (Hardhat)

```
cd charityflow-hardhat
npm install
npx hardhat compile
npx hardhat test
npx hardhat node          # Terminal 1
npx hardhat run scripts/deploy.js --network localhost   # Terminal 2
```

Sepolia Deployment

```
cd charityflow-hardhat
# Add to .env: PRIVATE_KEY, ALCHEMY_API_URL (eth-sepolia)
npm run deploy:sepolia
# Frontend .env is auto-updated with contract address
```

Frontend

```
cd charityflow-frontend
npm install
# Add VITE_PINATA_JWT to .env for IPFS uploads
npm run dev
```

Open http://localhost:5173. Connect MetaMask on Sepolia or Hardhat local.

8. Smart Contract API

Function	Access	Description
createCampaign(...)	Public	Create campaign
donate(campaignId, message)	Public (payable)	Donate ETH
withdrawFunds(...)	Charity	Withdraw with proof
submitProof(...)	Charity	Upload proof
postImpactReport(...)	Charity	Post impact update
cancelCampaign(campaignId)	Charity/Admin	Cancel campaign
claimRefund(donationId)	Donor	Refund on cancelled campaign
verifyCharity(address)	Admin	Verify charity
verifyProof(proofId)	Admin	Verify proof

9. User Guide

Donor: Connect MetaMask → Browse campaigns → Donate → Track via Fund Flow timeline and activity log.

Charity: Connect MetaMask → Charity Portal → Create Campaign (upload IPFS image) → Withdraw, Submit Proof, Post Impact Report.

Admin: Deployer address. Use Hardhat console or Remix for verifyCharity, verifyProof, withdrawPlatformFees.

10. Testing

```
cd charityflow-hardhat
npx hardhat test
```

41 unit tests across: Deployment, Campaign Creation, Donations, Proof of Work, Withdrawals, Refunds, Impact Reports, Admin Functions, Deadline Enforcement.

11. Deployment (Sepolia)

Contract address: 0xE9635eEE0b6EFeB423e9B7aE789C62DB41805F1d

Block: 10439368

Explorer: sepolia.etherscan.io

12. Known Issues & Limitations

1. MetaMask only — No WalletConnect
2. No pagination — Campaign list loads all
3. Admin UI — Verification via console/Remix only
4. Node version — Some packages prefer Node 20+

13. Future Improvements

1. The Graph subgraph for event indexing
2. Multi-token donations (USDC, DAI)
3. WalletConnect for mobile
4. DAO governance for charity verification
5. Mainnet deployment

14. Learning Outcomes Addressed

- **LO2:** Analyzed blockchain use case (charity transparency)
- **LO3:** Applied Ethereum architecture (Hardhat, Solidity, Ethers.js)
- **LO4:** Implemented smart contracts, transaction validation, events, on-chain ledger

15. References

- CryptoZombies, Hardhat Tutorial, Ethers.js Documentation
- MetaMask Developer Docs, Pinata IPFS, Sepolia Faucet
